Index

A

- abstractness of codebase, Abstractness and Instability
  - distance from main sequence, Distance from the Main Sequence
- ACID (atomicity, consistency, isolation, durability) transactions
  - about, Distributed Transactions-Distributed Transactions
    - distributed transactions versus, Distributed Transactions
    - example, Distributed Transactions
  - BASE transactions versus, Distributed Transactions
  - database types
    - cloud native databases, Cloud Native Databases
    - document databases, Document Databases
    - graph databases, Graph Databases
    - NewSQL databases, NewSQL Databases
    - relational databases, Relational Databases, Relational Databases
    - time series databases, Time-Series Databases
- afferent coupling, Afferent and Efferent Coupling, Abstractness and Instability
- aggregate orientation, Relational Databases
  - GraphQL read-only aggregated data, Strict Versus Loose Contracts
  - key-value databases, Key-Value Databases
  - NoSQL databases understanding, Column Family Databases
- agility
  - modularity providing, Architectural Modularity, Modularity Drivers
  - speed-to-market via, Modularity Drivers
  - testability for, Testability
  - versioning shared libraries, Versioning Strategies
- Amazon DynamoDB key-value database, Key-Value Databases
- Ambler, Scott, Decomposing Monolithic Data
- analytical data
  - about, The Importance of Data in Architecture, Managing Analytical Data
  - data meshes, The Data Mesh-When to Use Data Mesh

    - analytical reporting coupled, Data Mesh, Coupling, and Architecture Quantum
    - when to use, When to Use Data Mesh
  - definition, The Importance of Data in Architecture
  - domain over technical partitioning, The Data Lake
  - previous approaches
    - data lakes, The Data Lake-The Data Lake
    - data warehouses, The Data Warehouse-The Data Warehouse
  - Sysops Squad saga, Managing Analytical Data, The Data Warehouse, The Data Lake
    - data mesh, Sysops Squad Saga: Data Mesh-Sysops Squad Saga: Data Mesh
- annotations
  - distributed transaction management, Techniques for Managing Sagas
  - metadata for, Code Replication
- Anthology Saga pattern, Anthology Saga(aec) Pattern-Anthology Saga(aec) Pattern
- Apache Cassandra column family database, Column Family Databases
- Apache Ignite replicated caching, Replicated Caching Pattern
- APIs
  - endpoint versioning for shared services, Change Risk
  - platforms for code reuse, Reuse via Platforms
- Architectural Decision Records (ADRs), Architectural Decision Records
- architectural decomposition
  - about, Architectural Decomposition
    - eating the elephant, Architectural Decomposition
  - codebase analysis
    - about, Is the Codebase Decomposable?
    - abstractness and instability, Abstractness and Instability
    - afferent and efferent coupling, Afferent and Efferent Coupling
    - distance from main sequence, Distance from the Main Sequence
  - component-based, Component-Based Decomposition-Component-Based Decomposition
    - about, Architectural Decomposition, Component-Based Decomposition Patterns
    - about patterns, Component-Based Decomposition, Component-Based Decomposition Patterns
    - architecture stories, Component-Based Decomposition Patterns
    - create component domains, Create Component Domains Pattern-Fitness function: All namespaces under <root namespace node> should be restricted to <list of domains>

    - create domain services, Create Domain Services Pattern-Fitness function: All components in <some domain service> should start with the same namespace
    - determine component dependencies, Determine Component Dependencies Pattern-Fitness function: <some component> should not have a dependency on <another component>
    - flatten components, Flatten Components Pattern-Fitness function: No source code should reside in a root namespace
    - gather common domain components, Gather Common Domain Components Pattern-Fitness function: Find common code across components
    - identify and size components, Identify and Size Components Pattern-Sysops Squad Saga: Sizing Components
    - services built from components, Component-Based Decomposition
  - data
    - about, Pulling Apart Operational Data
    - about decomposing monolithic, Decomposing Monolithic Data
    - assign tables to data domains, Step 2: Assign Tables to Data Domains-Step 2: Assign Tables to Data Domains
    - create data domains, Step 1: Analyze Database and Create Data Domains
    - drivers of decomposition, Data Decomposition Drivers-Database transactions
    - Refactoring Databases (Ambler and Sadalage), Decomposing Monolithic Data
    - schemas to separate servers, Step 4: Move Schemas to Separate Database Servers
    - separate connections to data domains, Step 3: Separate Database Connections to Data Domains
    - switch over to independent servers, Step 5: Switch Over to Independent Database Servers
    - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
  - tactical forking, Component-Based Decomposition-Trade-Offs
    - about, Architectural Decomposition
    - trade-offs, Trade-Offs
  - volatility-based decomposition, Code Volatility
- architectural fitness functions (see fitness functions for governance)
- architecture
  - architecture governance, Architecture Fitness Functions
    - Big Ball of Mud anti-pattern, Is the Codebase Decomposable?
    - Equifax data breach, Using Fitness Functions
    - fitness functions, Using Fitness Functions
      - (see also fitness functions)

    - architecture quantum, Architecture (Quantum | Quanta)
        - (see also architecture quantum)
    - brittleness, Code Reuse: When Does It Add Value?
        - data warehouses, The Data Warehouse
        - strict contracts creating, Strict Versus Loose Contracts
    - composition versus inheritance, Shared Service
    - data importance, The Importance of Data in Architecture
        - (see also data)
    - definitions of terms, Architecture Versus Design: Keeping Definitions Simple
        - architecture definition, Why “The Hard Parts”?
    - design versus, Architecture Versus Design: Keeping Definitions Simple-Architecture Versus Design: Keeping Definitions Simple
        - security controlled through design, Sysops Squad Saga: Customer Registration Granularity
    - evolution of, Giving Timeless Advice About Software Architecture, Architecture Fitness Functions, Architectural Modularity
        - (see also modularity)
    - iterative nature of analysis, Analyze Coupling Points
    - why more important than how, Architecture Versus Design: Keeping Definitions Simple
- architecture governance, Architecture Fitness Functions
    - Big Ball of Mud anti-pattern, Is the Codebase Decomposable?
    - Equifax data breach, Using Fitness Functions
    - fitness functions, Using Fitness Functions
        - (see also fitness functions)
- architecture quantum
    - about, Architecture (Quantum | Quanta), Architectural quantum
    - coupling, static versus dynamic, Architecture (Quantum | Quanta)
        - dynamic quantum coupling, Dynamic Quantum Coupling-Coordination
        - high static coupling, High Static Coupling-High Static Coupling
        - user interface, High Static Coupling
    - data disintegration driver, Architectural quantum
        - separate databases, Step 4: Move Schemas to Separate Database Servers
        - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
    - data product quantum, Definition of Data Mesh-Data Product Quantum
    - high functional cohesion, High Functional Cohesion
        - database, Architectural quantum
    - independently deployable, Independently Deployable
    - Sysops Squad saga, Sysops Squad Saga: Understanding Quanta
- architecture stories, Component-Based Decomposition Patterns

- ArchUnit tool, Fitness function: All components in <some domain service> should start with the same namespace
- Aslett, Matthew, NewSQL Databases
- asynchronous communication
  - about complexity of, Fantasy Fiction Saga(aao) Pattern, Horror Story(aac) Pattern
  - definition, Architecture Versus Design: Keeping Definitions Simple
  - delegate technique of data ownership, Delegate Technique
  - distributed data access, Column Schema Replication Pattern
  - fault tolerance, Availability/Fault Tolerance
  - performance issues, Performance
    - messaging mitigating, Performance
  - publish-and-subscribe messaging, Event-Based Pattern
    - dead letter queue, Event-Based Pattern
    - durable subscribers, Event-Based Pattern
  - synchronous versus, Communication, Sysops Squad Saga: Understanding Quanta
- atomic fitness functions, Using Fitness Functions
- atomic workflow
  - consistency, Consistency
  - definition, Architecture Versus Design: Keeping Definitions Simple
  - eventual consistency versus, Architecture Versus Design: Keeping Definitions Simple, Consistency
- atomicity in ACID transactions, Distributed Transactions
  - distributed transactions versus, Distributed Transactions
- attributes for distributed transaction management, Techniques for Managing Sagas
- availability
  - basic availability of BASE transactions, Distributed Transactions
  - coupling relationship, Analyze Coupling Points
  - database types
    - about availability, Selecting a Database Type
    - cloud native databases, Cloud Native Databases
    - column family databases, Column Family Databases
    - document databases, Document Databases
    - graph databases, Graph Databases
    - key-value databases, Key-Value Databases
    - NewSQL databases, NewSQL Databases
    - relational databases, Relational Databases
    - time series databases, Time-Series Databases
  - modularity and, Availability/Fault Tolerance

- AWS Redshift cloud native database, Cloud Native Databases
- Azure CosmosDB cloud native database, Cloud Native Databases

B

- backpressure information link, Sysops Squad Saga: Ticket Assignment Granularity
- backup and restore databases, Step 4: Move Schemas to Separate Database Servers
- bandwidth and stamp coupling, Bandwidth
- BASE (basic availability, soft state, eventual consistency) transactions, Distributed Transactions
- Beck, Kent, Architecture Fitness Functions
- Berners-Lee, Tim, The Importance of Data in Architecture
- best practices nonexistent, What Happens When There Are No “Best Practices”?
- Big Ball of Mud anti-pattern, Is the Codebase Decomposable?
  - tactical forking, Component-Based Decomposition-Trade-Offs
- bounded context in microservices, The Importance of Data in Architecture, Discerning Coupling in Software Architecture, Change control
  - architecture quanta, High Static Coupling
  - architecture quantum versus, Sysops Squad Saga: Understanding Quanta
  - breaking database changes controlled, Change control-Change control
    - database abstraction, Change control-Change control
  - data domains, Decomposing Monolithic Data
    - combining data domains, Step 2: Assign Tables to Data Domains
  - data requirement in some architectures, Pulling Apart Operational Data
  - data sovereignty per service, Step 3: Separate Database Connections to Data Domains
  - granularity and, Data Relationships
  - high functional cohesion, High Functional Cohesion
  - ownership of data (see ownership of data)
- breaking changes to data structure, Change control
  - bounded context controlling, Change control-Change control
    - database abstraction, Change control-Change control
- brittleness in architecture, Code Reuse: When Does It Add Value?
  - data warehouses, The Data Warehouse
  - strict contracts creating, Strict Versus Loose Contracts
- broker style event-driven architecture quantum, High Static Coupling, Sysops Squad Saga: Understanding Quanta
- Brooks, Fred, What Happens When There Are No “Best Practices”?
- business case for modularity, Sysops Squad Saga: Creating a Business Case-Sysops Squad Saga: Creating a Business Case

C

- caching for distributed data access, Replicated Caching Pattern-Replicated Caching Pattern
- Chain of Responsibility design pattern, Time Travel Saga(sec) Pattern
- The Checklist Manifesto (Gawande), Using Fitness Functions
- choreographed coordination, Choreography Communication Style-Choreography Communication Style
  - about coordination, Coordination, Managing Distributed Workflows
  - about orchestrated versus, Managing Distributed Workflows, Phone Tag Saga(sac) Pattern
    - trade-offs, Trade-Offs Between Orchestration and Choreography
  - definition, Architecture Versus Design: Keeping Definitions Simple
  - service granularity and, Workflow and Choreography-Workflow and Choreography
  - stamp coupling for management, Stamp Coupling for Workflow Management
  - workflow state management, Workflow State Management-Workflow State Management
    - Front Controller pattern, Workflow State Management
    - stamp coupling, Workflow State Management
    - stateless choreography, Workflow State Management
- client/server applications, The Data Warehouse
- cloud native databases, Cloud Native Databases
- Cockburn, Alistair, Sidecars and Service Mesh
- CockroachDB NewSQL database, NewSQL Databases
- code replication as reuse pattern, Code Replication-When to Use
- code reuse patterns
  - about, Reuse Patterns
  - code replication, Code Replication-When to Use
  - granularity and shared code, Shared Code-Shared Code, Dependency Management and Change Control
  - orthogonal reuse pattern, Sidecars and Service Mesh
  - shared libraries
    - about, Shared Library
    - granularity and, Dependency Management and Change Control
    - versioning strategies, Versioning Strategies-When To Use
  - shared services, Shared Service-When to Use
    - change risk, Change Risk
    - fault tolerance, Fault Tolerance
    - performance, Performance

    - scalability, Scalability
    - versioning via API endpoints, Change Risk
  - sidecars and service mesh, Sidecars and Service Mesh-When to Use
    - Sysops Squad saga, Sysops Squad Saga: Common Infrastructure Logic
  - strict contracts and, Strict Versus Loose Contracts
  - Sysops Squad saga, Reuse Patterns
    - shared domain functionality, Sysops Squad Saga: Shared Domain Functionality
    - sidecars and service mesh, Sysops Squad Saga: Common Infrastructure Logic
  - value added, Code Reuse: When Does It Add Value?
    - reuse via platforms, Reuse via Platforms
- code volatility as granularity driver, Code Volatility
- cohesion
  - high functional cohesion, High Functional Cohesion
    - database architecture quantum, Architectural quantum
  - maintainability and, Maintainability
  - metrics information link, Service Scope and Function
  - service granularity, Service Scope and Function, Fault Tolerance
- column family databases, Column Family Databases
- communication
  - about synchronous versus asynchronous, Communication, Sysops Squad Saga: Understanding Quanta
  - analytical data mesh, Data Mesh, Coupling, and Architecture Quantum
  - coordination required, Coordination
  - database connection pool, Connection management
    - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
  - distributed data access, Column Schema Replication Pattern
  - dynamic coupling as, Pulling Things Apart, Architecture (Quantum | Quanta)
    - communication dependencies, Architecture (Quantum | Quanta)
    - saga pattern matrix, Coordination, Transactional Saga Patterns, Analyze Coupling Points
  - east-west communication, Workflow and Choreography-Workflow and Choreography
  - interservice communication, Workflow and Choreography-Workflow and Choreography
    - API endpoint versioning for shared services, Change Risk
    - bounded context data, Data Relationships
    - data ownership, Common Ownership Scenario, Delegate Technique
    - distributed data access, Interservice Communication Pattern
  - performance issues, Performance

    - latency with distributed data, Interservice Communication Pattern
  - publish-and-subscribe messaging, Event-Based Pattern
    - dead letter queue, Event-Based Pattern
    - durable subscribers, Event-Based Pattern
  - service scalability and elasticity, Scalability
- compensating updates in distributed transactions, Orchestrated Request-Based Pattern, Epic Saga(sao) Pattern
  - state management instead, State Management and Eventual Consistency, Saga State Machines
  - Sysops Squad saga, Sysops Squad Saga: Atomic Transactions and Compensating Updates-Sysops Squad Saga: Atomic Transactions and Compensating Updates
- competitive advantage as modularity driver, Modularity Drivers
- compile-based versus runtime changes, Change Risk
- component-based decomposition, Component-Based Decomposition-Component-Based Decomposition
  - about, Architectural Decomposition, Component-Based Decomposition Patterns
  - about components, Component-Based Decomposition
  - patterns
    - about, Component-Based Decomposition, Component-Based Decomposition Patterns
    - architecture stories, Component-Based Decomposition Patterns
    - create component domains, Create Component Domains Pattern-Fitness function: All namespaces under <root namespace node> should be restricted to <list of domains>
    - create domain services, Create Domain Services Pattern-Fitness function: All components in <some domain service> should start with the same namespace
    - determine component dependencies, Determine Component Dependencies Pattern-Fitness function: <some component> should not have a dependency on <another component>
    - flatten components, Flatten Components Pattern-Fitness function: No source code should reside in a root namespace
    - gather common domain components, Gather Common Domain Components Pattern-Fitness function: Find common code across components
    - identify and size components, Identify and Size Components Pattern-Sysops Squad Saga: Sizing Components
- components
  - Create Component Domains pattern, Create Component Domains Pattern-Fitness function: All namespaces under <root namespace node> should be re-

stricted to <list of domains>
    - fitness functions, Fitness Functions for Governance
  - definition, Architecture Versus Design: Keeping Definitions Simple, Maintainability, Component-Based Decomposition, Pattern Description
  - dependencies determination, Pattern Description
  - Identify and Size Components pattern, Identify and Size Components Pattern-Sysops Squad Saga: Sizing Components
    - fitness functions, Fitness Functions for Governance-Fitness function: No component shall exceed <some number of standard deviations> from the mean component size
  - maintainability and, Maintainability
  - names, Pattern Description
  - namespaces, Pattern Description
  - size determination, Pattern Description
    - size and granularity, Service Scope and Function
- composite architecture characteristics, Using Fitness Functions
- composition versus inheritance in design, Shared Service
- connection management
  - connection quotas, Connection management
  - data disintegration driver, Connection management-Connection management
  - scalability includes connections, Scalability
    - database per service, Scalability
    - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
- consistency
  - about, Consistency, Selecting a Database Type
  - ACID transactions, Distributed Transactions
    - distributed transactions versus, Distributed Transactions
  - database types
    - cloud native ACID, Cloud Native Databases
    - column family tunable, Column Family Databases
    - document ACID, Document Databases
    - graph ACID, Graph Databases
    - key-value tunable, Key-Value Databases
    - NewSQL ACID, NewSQL Databases
    - relational ACID, Relational Databases
    - time series ACID or tunable, Time-Series Databases
  - dynamic coupling relationship matrix, Coordination, Transactional Saga Patterns, Analyze Coupling Points
  - eventual consistency
    - about patterns, Eventual Consistency Patterns

    - atomic workflow versus, Architecture Versus Design: Keeping Definitions Simple, Consistency
    - background synchronization pattern, Background Synchronization Pattern-Background Synchronization Pattern
    - BASE transactions, Distributed Transactions
    - event-based pattern, Event-Based Pattern-Event-Based Pattern
    - finite state machines for, State Management and Eventual Consistency
    - orchestrated request-based pattern, Orchestrated Request-Based Pattern-Orchestrated Request-Based Pattern
  - majority quorum, Key-Value Databases, Document Databases
  - tunable consistency, Key-Value Databases, Column Family Databases
- Constantine, Larry, Afferent and Efferent Coupling, Putting Things Back Together
- consumer-driven contracts, Consumer-driven contracts
- context for trade-off analysis, The “Out-of-Context” Trap-The “Out-of-Context” Trap
- contracts
  - about, Contracts
  - bounded context data ownership, Change control-Change control
  - contract fidelity and microservices, Coupling levels
    - consumer-driven contracts, Consumer-driven contracts
  - definition, Architecture Versus Design: Keeping Definitions Simple
  - distributed data access
    - Data Domain not requiring, Data Domain Pattern
    - Interservice Communication requiring, Interservice Communication Pattern, Data Domain Pattern
    - Replicated Cache requiring, Data Domain Pattern
  - high static coupling, High Static Coupling
  - loose contracts, Strict Versus Loose Contracts
    - decoupling via, Strict Versus Loose Contracts, Loose contracts
    - fitness functions required, Loose contracts
    - trade-offs, Loose contracts
  - microservice contracts, Contracts in Microservices
  - stamp coupling
    - about, Stamp Coupling
    - bandwidth, Bandwidth
    - over-coupling via, Over-Coupling via Stamp Coupling
    - workflow management, Stamp Coupling for Workflow Management
  - strict contracts, Strict Versus Loose Contracts
    - JSON example, Strict Versus Loose Contracts
    - trade-offs, Strict contracts

    - Sysops Squad saga, Contracts, Sysops Squad Saga: Managing Ticketing Contracts
- cooperative quantum, Data Product Quantum
- coordination of workflow
    - about, Coordination, Managing Distributed Workflows
    - about trade-offs, Trade-Offs Between Orchestration and Choreography
    - choreographed coordination, Choreography Communication Style-Choreography Communication Style
        - about orchestration versus, Managing Distributed Workflows, Phone Tag Saga(sac) Pattern
        - definition, Architecture Versus Design: Keeping Definitions Simple
        - stamp coupling for management, Stamp Coupling for Workflow Management
        - workflow state management, Workflow State Management
    - dynamic coupling relationship matrix, Coordination, Transactional Saga Patterns, Analyze Coupling Points
    - orchestrated coordination, Orchestration Communication Style-Orchestration Communication Style
        - about choreographed versus, Managing Distributed Workflows, Phone Tag Saga(sac) Pattern
        - definition, Architecture Versus Design: Keeping Definitions Simple
        - workflow state management, Workflow State Management
    - stamp coupling for management, Stamp Coupling for Workflow Management
    - Sysops Squad saga, Managing Distributed Workflows, Sysops Squad Saga: Managing Workflows-Sysops Squad Saga: Managing Workflows
- coupling
    - about static versus dynamic, Architecture (Quantum | Quanta)
        - dynamic coupling, Pulling Things Apart, Architecture (Quantum | Quanta)
        - static coupling, Pulling Things Apart, Architecture (Quantum | Quanta)
        - Sysops Squad saga, Sysops Squad Saga: Understanding Quanta
    - architectural decomposition
        - afferent coupling, Afferent and Efferent Coupling, Abstractness and Instability
        - common domain functionality consolidated, Sysops Squad Saga: Gathering Common Components
        - determining component dependencies, Pattern Description
        - efferent coupling, Afferent and Efferent Coupling, Abstractness and Instability
    - architecture quantum, Architecture (Quantum | Quanta), High Static Coupling
        - dynamic quantum coupling, Dynamic Quantum Coupling-Coordination

    - high static coupling, High Static Coupling-High Static Coupling
    - independently deployable, Independently Deployable
    - Sysops Squad saga, Sysops Squad Saga: Understanding Quanta
    - user interface, High Static Coupling
  - availability relationship, Analyze Coupling Points
  - data relationships, Data relationships
  - definition, Architecture Versus Design: Keeping Definitions Simple, Discerning Coupling in Software Architecture
  - interservice communication pattern, Interservice Communication Pattern
  - loose contracts for decoupling, Strict Versus Loose Contracts, Loose contracts
    - consumer-driven contracts, Consumer-driven contracts
    - microservices, Coupling levels
    - microservices and contract fidelity, Coupling levels
    - strict contract tight coupling, Strict contracts
  - maintainability and, Maintainability
  - operational coupling, Sidecars and Service Mesh-When to Use
    - Sysops Squad saga, Sysops Squad Saga: Common Infrastructure Logic
  - orchestration, Orchestration Communication Style
    - choreography versus, Phone Tag Saga(sac) Pattern
  - orthogonal coupling, Sidecars and Service Mesh
  - scaling relationship, Phone Tag Saga(sac) Pattern, Analyze Coupling Points
  - semantic coupling, Choreography Communication Style
    - implementation coupling, Choreography Communication Style
    - stamp coupling, Stamp Coupling for Workflow Management
  - stamp coupling
    - bandwidth, Bandwidth
    - choreography state management, Workflow State Management
    - over-coupling via, Over-Coupling via Stamp Coupling
    - workflow management, Stamp Coupling for Workflow Management
  - trade-off analysis, Discerning Coupling in Software Architecture-Discerning Coupling in Software Architecture, Coupling-Analyze Coupling Points
    - coupling points, Analyze Coupling Points
    - static coupling diagram, Coupling
- Create Component Domains pattern, Create Component Domains Pattern-Fitness function: All namespaces under <root namespace node> should be restricted to <list of domains>
  - fitness functions, Fitness Functions for Governance
  - Sysops Squad saga, Sysops Squad Saga: Creating Component Domains-Sysops Squad Saga: Creating Component Domains, Sysops Squad Saga: Creating Domain Services

- Create Domain Services pattern, Create Domain Services Pattern-Fitness function: All components in <some domain service> should start with the same namespace
  - fitness functions, Fitness Functions for Governance
  - Sysops Squad saga, Sysops Squad Saga: Creating Domain Services
- credit card data (see PCI (Payment Card Industry) data)
- custom attributes for distributed transaction management, Techniques for Managing Sagas
- cyclomatic complexity and maintainability, Maintainability

D

- data
  - about importance of, The Importance of Data in Architecture
  - about operational versus analytical, The Importance of Data in Architecture
  - analytical data
    - about, The Importance of Data in Architecture, Managing Analytical Data
    - data lakes, The Data Lake-The Data Lake
    - data meshes, The Data Mesh-When to Use Data Mesh
    - data meshes coupled to reporting, Data Mesh, Coupling, and Architecture Quantum
    - data warehouses, The Data Warehouse-The Data Warehouse
    - definition, The Importance of Data in Architecture
    - domain over technical partitioning, The Data Lake
    - Sysops Squad saga, Managing Analytical Data, The Data Warehouse, The Data Lake, Sysops Squad Saga: Data Mesh-Sysops Squad Saga: Data Mesh
  - breaking changes, Change control
    - bounded contexts controlling, Change control-Change control
  - coupling
    - microservices, High Static Coupling
    - multiple quanta event-driven architecture, High Static Coupling
    - relationships among data, Data relationships
  - data domains (see data domains)
  - database-per-service, Scalability
    - breaking apart database to achieve, Data relationships
  - decomposition of operational data
    - about, Pulling Apart Operational Data
    - about decomposing monolithic, Decomposing Monolithic Data
    - assign tables to data domains, Step 2: Assign Tables to Data Domains-Step 2: Assign Tables to Data Domains
    - create data domains, Step 1: Analyze Database and Create Data Domains

    - data access in other domains, Step 3: Separate Database Connections to Data Domains
    - database-per-service requiring, Data relationships
    - dependencies, Decomposing Monolithic Data-Decomposing Monolithic Data
    - drivers of, Data Decomposition Drivers-Database transactions
    - Refactoring Databases (Ambler and Sadalage), Decomposing Monolithic Data
    - schemas to separate servers, Step 4: Move Schemas to Separate Database Servers
    - separate connections to data domains, Step 3: Separate Database Connections to Data Domains
    - switch over to independent servers, Step 5: Switch Over to Independent Database Servers
    - Sysops Squad saga, Pulling Apart Operational Data, Sysops Squad Saga: Justifying Database Decomposition
- disintegration drivers
    - about, Data Disintegrators
    - architecture quantum, Architectural quantum
    - connection management, Connection management-Connection management
    - database change control, Change control-Change control
    - database type optimization, Database type optimization
    - fault tolerance, Fault tolerance
    - scalability, Scalability
- distributed data access (see distributed data access)
- granularity and data relationships, Data Relationships-Data Relationships
- integration drivers
    - about, Data Integrators
    - database transactions, Database transactions
    - relationships, Data relationships
- latency with distributed data, Interservice Communication Pattern
- ownership
    - about, Data Ownership and Distributed Transactions
    - assigning, Assigning Data Ownership
    - common ownership, Common Ownership Scenario
    - data mesh domain ownership, Definition of Data Mesh
    - data sovereignty per service, Step 3: Separate Database Connections to Data Domains
    - distributed data access, Column Schema Replication Pattern, Replicated Caching Pattern
    - joint ownership, Joint Ownership Scenario-Delegate Technique

    - service consolidation technique, Service Consolidation Technique
    - single ownership, Single Ownership Scenario
    - summary, Data Ownership Summary
    - Sysops Squad saga, Data Ownership and Distributed Transactions, Sysops Squad Saga: Data Ownership for Ticket Processing-Sysops Squad Saga: Data Ownership for Ticket Processing
  - security
    - credit card data, Security
    - granularity driver, Security
  - services sharing, Pulling Apart Operational Data
    - changes to database, Change control
    - connection management, Connection management-Connection management
    - single point of failure, Fault tolerance
  - single point of failure vulnerability, Fault tolerance
    - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
  - Sysops Squad data model, Sysops Squad Data Model
    - data domains, Decomposing Monolithic Data
- data domain technique of joint ownership, Data Domain Technique-Data Domain Technique
- data domains
  - about, Decomposing Monolithic Data
  - about decomposing monolithic data, Decomposing Monolithic Data
    - assign tables to data domains, Step 2: Assign Tables to Data Domains-Step 2: Assign Tables to Data Domains
    - bounded context rules, Decomposing Monolithic Data
    - create data domains, Step 1: Analyze Database and Create Data Domains
    - schemas to separate servers, Step 4: Move Schemas to Separate Database Servers
    - separate connections to data domains, Step 3: Separate Database Connections to Data Domains
    - switch over to independent servers, Step 5: Switch Over to Independent Database Servers
  - combining
    - service consolidation, Service Consolidation Technique
    - tables tightly coupled, Step 2: Assign Tables to Data Domains
  - data access in other domains, Step 3: Separate Database Connections to Data Domains
  - data schemas versus, Step 2: Assign Tables to Data Domains
    - synonyms for tables, Step 2: Assign Tables to Data Domains

    - distributed data access, Data Domain Pattern-Data Domain Pattern
    - joint ownership of data, Data Domain Technique-Data Domain Technique
    - soccer ball visualization, Decomposing Monolithic Data
    - Sysops Squad data model, Decomposing Monolithic Data
- data lakes, The Data Lake-The Data Lake
- Data Mesh (Dehghani), Definition of Data Mesh
- data meshes
    - about, The Data Mesh
    - Data Mesh (Dehghani), Definition of Data Mesh
    - data product quantum, Definition of Data Mesh-Data Product Quantum
    - Sysops Squad saga, Sysops Squad Saga: Data Mesh-Sysops Squad Saga: Data Mesh
    - when to use, When to Use Data Mesh
- data product quantum (DPQ), Definition of Data Mesh-Data Product Quantum
    - aggregate DPQ, Data Product Quantum
    - cooperative quantum, Data Product Quantum
    - fit-for-purpose DPQ, Data Product Quantum
    - source-aligned (native) DPQ, Data Product Quantum
- data warehouses, The Data Warehouse-The Data Warehouse
    - Star Schema, The Data Warehouse, The Data Warehouse
    - technical partitioning, The Data Warehouse
- database abstraction via bounded context, Change control-Change control
- Database as a Service (DBaaS), Relational Databases
    - NewSQL databases, NewSQL Databases
- database refactoring book, Decomposing Monolithic Data
- database structure Sysops Squad saga, Sysops Squad Saga: Polyglot Databases-Sysops Squad Saga: Polyglot Databases
- database transactions and granularity, Database Transactions
- database types, Time-Series Databases
    - about characteristics rated, Selecting a Database Type
    - aggregate orientation, Relational Databases
    - cloud native databases, Cloud Native Databases
    - column family databases, Column Family Databases
    - database type optimization, Database type optimization
    - document databases, Document Databases
    - graph databases, Graph Databases-Graph Databases
    - key-value databases, Key-Value Databases-Key-Value Databases
    - NewSQL databases, NewSQL Databases
    - NoSQL databases, Key-Value Databases
    - relational databases, Relational Databases-Relational Databases

    - schema-less databases, Document Databases
    - Sysops Squad saga, Sysops Squad Saga: Polyglot Databases
    - time series databases, Time-Series Databases-Time-Series Databases
- database-per-service, Scalability
    - breaking apart database to achieve, Data relationships
- databases separated physically, Step 4: Move Schemas to Separate Database Servers
- Datomic cloud native database, Cloud Native Databases
- DDD (see Domain-Driven Design)
- De La Torre, Fausto, Tactical Forking
- dead letter queue (DLQ), Event-Based Pattern
- decision making, Discerning Coupling in Software Architecture-Discerning Coupling in Software Architecture
    - (see also trade-off analysis)
- decomposition (see architectural decomposition)
- Decorator Design Pattern, When to Use
- Dehghani, Zhamak, The Data Mesh, Definition of Data Mesh
- delegate technique of joint data ownership, Delegate Technique-Delegate Technique
- dependencies
    - Determine Component Dependencies pattern, Determine Component Dependencies Pattern-Fitness function: <some component> should not have a dependency on <another component>
    - dynamic coupling as communication dependencies, Architecture (Quantum | Quanta)
    - monolithic data domains, Decomposing Monolithic Data-Decomposing Monolithic Data
        - Refactoring Databases (Ambler and Sadalage), Decomposing Monolithic Data
        - synonyms for tables, Step 2: Assign Tables to Data Domains
    - shared libraries, Dependency Management and Change Control, Versioning Strategies
    - static coupling as operational dependencies, Pulling Things Apart, Architecture (Quantum | Quanta)
    - tools
        - JDepend, Using Fitness Functions, Afferent and Efferent Coupling
        - NetArchTest, Using Fitness Functions
        - visualization importance, Pattern Description
        - visualization tools resource, Pattern Description
- deployment
    - architectural modularity described, Testability

    - deployability in modularity, Deployability
    - fault tolerance, Availability/Fault Tolerance
  - cloud native Production Topology, Cloud Native Databases
  - independent deployability, Independently Deployable
- deprecation strategies for shared libraries, Versioning Strategies
- design
  - architecture versus, Architecture Versus Design: Keeping Definitions Simple-Architecture Versus Design: Keeping Definitions Simple
    - security controlled through design, Sysops Squad Saga: Customer Registration Granularity
  - Chain of Responsibility, Time Travel Saga(sec) Pattern
  - composition versus inheritance, Shared Service
  - Decorator Design Pattern, When to Use
  - Pipes and Filters, Time Travel Saga(sec) Pattern, Anthology Saga(aec) Pattern
- Determine Component Dependencies pattern, Determine Component Dependencies Pattern-Fitness function: <some component> should not have a dependency on <another component>
  - fitness functions, Fitness Functions for Governance
  - Sysops Squad saga, Sysops Squad Saga: Identifying Component Dependencies-Sysops Squad Saga: Identifying Component Dependencies
- dimensional models of Star Schema, The Data Warehouse
- distance from main sequence, Distance from the Main Sequence
  - zones of uselessness and pain, Distance from the Main Sequence
- distributed architectures
  - bounded context in microservices, Discerning Coupling in Software Architecture
    - (see also bounded context)
  - code reuse patterns, Reuse Patterns
    - (see also code reuse patterns)
  - data importance, The Importance of Data in Architecture
    - (see also data)
  - data sovereignty per service, Step 3: Separate Database Connections to Data Domains
  - database connection pool, Connection management
    - connection management, Connection management
    - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
  - dynamic coupling matrix, Coordination, Transactional Saga Patterns, Analyze Coupling Points
  - eventual consistency, Eventual Consistency Patterns
    - (see also eventual consistency)
  - evolution of architecture, Giving Timeless Advice About Software Architecture

  - sagas as transactional behavior, Introducing the Sysops Squad Saga
    - (see also sagas)
  - sharing data, Data Domain Technique
  - side effects, Sysops Squad Saga: Atomic Transactions and Compensating Updates
  - state machines, Saga State Machines-Saga State Machines
  - static coupling, High Static Coupling
    - multiple quanta, High Static Coupling
    - single quantum, High Static Coupling
  - transactionality challenges, Consistency
- distributed data access
  - Column Schema Replication pattern, Column Schema Replication Pattern
  - Data Domain pattern, Data Domain Pattern-Data Domain Pattern
  - Interservice Communication pattern, Interservice Communication Pattern
  - Replicated Caching pattern, Replicated Caching Pattern-Replicated Caching Pattern
  - Sysops Squad saga, Distributed Data Access, Sysops Squad Saga: Data Access for Ticket Assignment-Sysops Squad Saga: Data Access for Ticket Assignment
- distributed transactions
  - about, Distributed Transactions, Epic Saga(sao) Pattern
  - ACID transactions, Distributed Transactions-Distributed Transactions
    - example, Distributed Transactions
  - BASE transactions, Distributed Transactions
  - compensating updates, Orchestrated Request-Based Pattern, Epic Saga(sao) Pattern
    - Sysops Squad saga, Sysops Squad Saga: Atomic Transactions and Compensating Updates-Sysops Squad Saga: Atomic Transactions and Compensating Updates
  - eventual consistency, Eventual Consistency Patterns
    - (see also eventual consistency)
  - managing sagas, Techniques for Managing Sagas
    - (see also transactional saga patterns)
  - Sysops Squad sagas, Sysops Squad Saga: Data Ownership for Ticket Processing-Sysops Squad Saga: Data Ownership for Ticket Processing, Sysops Squad Saga: Atomic Transactions and Compensating Updates-Sysops Squad Saga: Atomic Transactions and Compensating Updates
- distributed workflows
  - about, Managing Distributed Workflows
  - about coordination, Coordination, Managing Distributed Workflows
  - about trade-offs, Trade-Offs Between Orchestration and Choreography

    - choreography, Choreography Communication Style-Choreography Communication Style
        - about orchestration versus, Managing Distributed Workflows
        - definition, Architecture Versus Design: Keeping Definitions Simple
        - workflow state management, Workflow State Management-Workflow State Management
    - orchestration, Orchestration Communication Style-Orchestration Communication Style
        - about choreographed versus, Managing Distributed Workflows
        - definition, Architecture Versus Design: Keeping Definitions Simple
        - workflow state management, Workflow State Management
    - semantic coupling, Choreography Communication Style
        - implementation coupling, Choreography Communication Style
    - service granularity and workflow, Workflow and Choreography-Workflow and Choreography
    - stamp coupling for management, Stamp Coupling for Workflow Management
    - Sysops Squad saga, Managing Distributed Workflows, Sysops Squad Saga: Managing Workflows-Sysops Squad Saga: Managing Workflows
    - transactional saga patterns, Transactional Sagas
        - (see also transactional saga patterns)
- document databases, Document Databases
- documentation for managing distributed transactions, Code Replication, Techniques for Managing Sagas
- domain partitioned architecture, Modularity Drivers
    - data domains, Decomposing Monolithic Data
    - data lakes losing relationships, The Data Lake
    - data mesh domain ownership, Definition of Data Mesh
    - granularity and shared domain functionality, Shared Code
    - maintainability and, Maintainability
    - technical partitioning
        - data warehouses as, The Data Warehouse
        - domain preferred, The Data Lake
        - domain workflows, Choreography Communication Style
- domain services
    - about, Create Domain Services Pattern
    - component domains into, Pattern Description
        - Create Domain Services pattern, Create Domain Services Pattern-Fitness function: All components in <some domain service> should start with the same namespace

    - fitness function for namespace, Fitness function: All components in <some domain service> should start with the same namespace
  - domain cohesion and granularity, Fault Tolerance
  - Front Controller in choreography, Workflow State Management
  - service-based architecture definition, Component-Based Decomposition, Create Domain Services Pattern
- domain versus infrastructure shared functionality, Pattern Description
- Domain-Driven Design (DDD)
  - bounded context, The Importance of Data in Architecture
    - architecture quantum, Sysops Squad Saga: Understanding Quanta
    - high functional cohesion and, High Functional Cohesion
  - information link, Pattern Description
  - namespaces from applications prior to, Pattern Description
- durability in ACID transactions, Distributed Transactions
  - distributed transactions versus, Distributed Transactions
- durable subscribers in pub/sub messaging, Event-Based Pattern
- dynamic coupling, Pulling Things Apart, Architecture (Quantum | Quanta)
  - about dimensions of, Managing Distributed Workflows, Coupling
  - analytical data communication pattern, Data Mesh, Coupling, and Architecture Quantum
  - communication, Communication, Sysops Squad Saga: Understanding Quanta
  - consistency, Consistency
  - coordination, Coordination
  - relationships among factors, Coordination, Transactional Saga Patterns
    - (see also sagas)
  - static versus, Architecture (Quantum | Quanta)
    - Sysops Squad saga, Sysops Squad Saga: Understanding Quanta

E

- east-west communication, Workflow and Choreography-Workflow and Choreography
- efferent coupling, Afferent and Efferent Coupling, Abstractness and Instability
- elasticity
  - coupling, Sysops Squad Saga: Understanding Quanta
  - definition, Scalability
  - mean time to startup and, Scalability
  - scalability versus, Sysops Squad Saga: Understanding Quanta, Scalability
- Enterprise Integration Patterns (Hohpe and Woolf), Step 1: Analyze Database and Create Data Domains

- enterprise service bus (ESB) global orchestrator, Orchestration Communication Style
- Epic Saga pattern, Epic Saga(sao) Pattern-Epic Saga(sao) Pattern
- Equifax data breach, Using Fitness Functions
- Evans, Eric, Relational Databases
- event-based eventual consistency, Event-Based Pattern-Event-Based Pattern
  - publish-and-subscribe messaging, Event-Based Pattern
    - dead letter queue, Event-Based Pattern
    - durable subscribers, Event-Based Pattern
- event-driven architecture (EDA)
  - multiple architecture quanta, High Static Coupling
  - single architecture quantum
    - broker style as, High Static Coupling, Sysops Squad Saga: Understanding Quanta
    - mediator style as, High Static Coupling
- eventual consistency
  - about consistency, Consistency
  - about patterns, Eventual Consistency Patterns
  - atomic workflow versus, Architecture Versus Design: Keeping Definitions Simple, Consistency
  - background synchronization pattern, Background Synchronization Pattern-Background Synchronization Pattern
  - BASE transactions, Distributed Transactions
  - event-based pattern, Event-Based Pattern-Event-Based Pattern
    - publish-and-subscribe messaging, Event-Based Pattern
  - finite state machines for, State Management and Eventual Consistency
  - orchestrated request-based pattern, Orchestrated Request-Based Pattern-Orchestrated Request-Based Pattern
- extensibility as granularity driver, Extensibility
- eXtreme Programming (XP), Architecture Fitness Functions

F

- Fairy Tale Saga pattern, Fairy Tale Saga(seo) Pattern-Fairy Tale Saga(seo) Pattern
- Fantasy Fiction Saga pattern, Fantasy Fiction Saga(aao) Pattern-Fantasy Fiction Saga(aao) Pattern
- fault tolerance
  - choreography, Workflow State Management
  - data disintegration driver, Fault tolerance
  - definition, Availability/Fault Tolerance, Fault tolerance, Fault Tolerance

    - granularity disintegration driver, Fault Tolerance
    - granularity integration driver, Workflow and Choreography-Workflow and Choreography
    - modularity driver, Modularity Drivers, Availability/Fault Tolerance
    - orchestration, Orchestration Communication Style
    - shared libraries, Fault Tolerance
    - shared services, Fault Tolerance
    - single point of failure, Fault tolerance
        - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
- fitness functions for governance
    - about, Using Fitness Functions-Using Fitness Functions
        - atomic, Using Fitness Functions
        - composite architecture characteristics, Using Fitness Functions
        - holistic, Using Fitness Functions
        - manual versus automated, Using Fitness Functions
        - outcomes, Using Fitness Functions
        - overuse avoided, Using Fitness Functions
    - dynamic coupling monitored, Architecture (Quantum | Quanta)
    - examples
        - component cycles avoided, Using Fitness Functions
        - layer relationship verification, Using Fitness Functions
    - loose coupling requiring, Loose contracts
    - patterns for component-based decomposition
        - create component domains, Fitness Functions for Governance
        - create domain services, Fitness Functions for Governance
        - determine component dependencies, Fitness Functions for Governance
        - flatten components, Fitness Functions for Governance
        - gather common domain components, Fitness Functions for Governance
        - identify and size components, Fitness Functions for Governance-Fitness function: No component shall exceed <some number of standard deviations> from the mean component size
    - zero-day exploits, Using Fitness Functions
- Flatten Components pattern, Flatten Components Pattern-Fitness function: No source code should reside in a root namespace
    - fitness functions, Fitness Functions for Governance
    - metrics for shared components, Pattern Description
    - Sysops Squad saga, Sysops Squad Saga: Flattening Components-Sysops Squad Saga: Flattening Components
- Foote, Brian, Is the Codebase Decomposable?
- Ford, Neal, Preface, Concept and Term References

- Fowler, Martin, Key-Value Databases, Shared Service
- Front Controller in choreography, Workflow State Management
- Fundamentals of Software Architecture (Richards and Ford), Preface, Concept and Term References-Trade-Off References

G

- Gather Common Domain Components pattern, Gather Common Domain Components Pattern-Fitness function: Find common code across components
  - fitness functions, Fitness Functions for Governance
  - Sysops Squad saga, Sysops Squad Saga: Gathering Common Components-Sysops Squad Saga: Gathering Common Components
- Gawande, Atul, Using Fitness Functions
- Gee, Trisha, Code Replication
- granularity
  - balancing trade-offs, Finding the Right Balance
  - definition, Scalability
  - disintegrators of granularity
    - about, Service Granularity, Granularity Disintegrators
    - code volatility, Code Volatility
    - extensibility, Extensibility
    - fault tolerance, Fault Tolerance
    - scalability and throughput, Scalability and Throughput
    - security, Security
    - service scope and function, Service Scope and Function
  - elasticity as function of, Scalability
  - integrators of granularity
    - about, Service Granularity, Granularity Integrators
    - data relationships, Data Relationships-Data Relationships
    - database transactions, Database Transactions
    - shared code, Shared Code-Shared Code
    - workflow and choreography, Workflow and Choreography-Workflow and Choreography
  - metrics, Service Granularity
  - microservice single responsibility, Service Granularity, Service Scope and Function
  - modularity versus, Service Granularity
  - shared data and, Data Domain Technique
  - shared libraries, Dependency Management and Change Control

    - Sysops Squad saga, Service Granularity, Sysops Squad Saga: Ticket Assignment Granularity-Sysops Squad Saga: Customer Registration Granularity
- graph databases, Graph Databases-Graph Databases
- GraphQL loose contracts, Strict Versus Loose Contracts
- Gremlin database language, Graph Databases
- gRPC protocol
    - information link, Change Risk
    - performance mitigation by, Performance
    - strict contracts by default, Strict Versus Loose Contracts

H

- hard parts of architecture, Why “The Hard Parts”?
- hash table data structure, Key-Value Databases
- Hello2morrow, Maintainability
- Henney, Kevlin, Code Replication
- Hexagonal Architecture, Sidecars and Service Mesh
- high functional cohesion, High Functional Cohesion
    - database architecture quantum, Architectural quantum
- Hohpe, Gregor, Step 1: Analyze Database and Create Data Domains, Sysops Squad Saga: Atomic Transactions and Compensating Updates
- Horror Story pattern, Horror Story(aac) Pattern-Horror Story(aac) Pattern

I

- Identify and Size Components pattern, Identify and Size Components Pattern-Sysops Squad Saga: Sizing Components
    - fitness functions, Fitness Functions for Governance-Fitness function: No component shall exceed <some number of standard deviations> from the mean component size
    - Sysops Squad saga, Component-Based Decomposition Patterns
- implementation coupling, Choreography Communication Style
- independent deployability, Independently Deployable
- InfluxDB time series database, Time-Series Databases
- infrastructure versus domain shared functionality, Pattern Description
- inheritance versus composition in design, Shared Service
- instability of codebase, Abstractness and Instability
    - distance from main sequence, Distance from the Main Sequence
- interservice communication, Workflow and Choreography-Workflow and Choreography

    - API endpoint versioning for shared services, Change Risk
    - bounded context data, Data Relationships
    - data ownership
        - common ownership, Common Ownership Scenario
        - delegate technique of joint ownership, Delegate Technique
    - distributed data access, Interservice Communication Pattern
- isolation in ACID transactions, Distributed Transactions
    - distributed transactions versus, Distributed Transactions
    - importance of, Sysops Squad Saga: Atomic Transactions and Compensating Updates
- isomorphic diagrams, Transactional Saga Patterns

J

- Java Message Service (Richards), Performance
- JDepend tool, Using Fitness Functions
    - Eclipse plug-in, Afferent and Efferent Coupling
- joint ownership of data
    - about, Joint Ownership Scenario
    - data domain technique, Data Domain Technique-Data Domain Technique
    - delegate technique, Delegate Technique-Delegate Technique
    - service consolidation technique, Service Consolidation Technique
    - table split technique, Table Split Technique-Table Split Technique

K

- key-value databases, Key-Value Databases-Key-Value Databases
- Kx time series database, Time-Series Databases

L

- latency in communication, Interservice Communication Pattern
- LATEST caution for versioning, Versioning Strategies
- layered architecture
    - architecture governance, Using Fitness Functions
    - modular architecture
        - maintainability, Maintainability
        - scalability and elasticity, Scalability
        - testability, Testability
- loose contracts
    - decoupling via, Strict Versus Loose Contracts

- fitness functions required, Loose contracts
- REST and GraphQL, Strict Versus Loose Contracts
- trade-offs, Loose contracts

M

- maintainability of modular architecture, Maintainability-Maintainability
  - metric for tracking, Maintainability
- majority quorum, Key-Value Databases, Document Databases
- Martin, Robert C., Abstractness and Instability, Service Scope and Function
- mean time to startup and elasticity, Scalability
- MECE (Mutually Exclusive, Collectively Exhaustive) lists, MECE Lists
- mediator in orchestration, Orchestration Communication Style
- mediator style event-driven architecture quantum, High Static Coupling
- MemcacheDB key-value database, Key-Value Databases
- messaging asynchronicity, Performance
  - Java Message Service (Richards), Performance
- metadata annotations, Code Replication
- microkernel architecture, Modularity Drivers
- microservices
  - architecture quantum, Architecture (Quantum | Quanta), Independently Deployable
    - decoupled services, High Static Coupling
    - high functional cohesion, High Functional Cohesion
    - user interface coupling, High Static Coupling
  - bounded context, The Importance of Data in Architecture, Discerning Coupling in Software Architecture
    - architecture quanta, High Static Coupling
    - architecture quantum versus, Sysops Squad Saga: Understanding Quanta
    - breaking database changes controlled, Change control-Change control
    - data domains, Decomposing Monolithic Data
    - data domains combined, Step 2: Assign Tables to Data Domains
    - data requirement, Pulling Apart Operational Data
    - database abstraction, Change control-Change control
    - granularity and, Data Relationships
    - high functional cohesion, High Functional Cohesion
    - ownership of data (see ownership of data)
  - contracts
    - about, Contracts in Microservices
    - consumer-driven contracts, Consumer-driven contracts

    - contract fidelity, Coupling levels
    - coupling levels, Coupling levels
  - coordination, Coordination
    - (see also coordination)
  - definition, Service Scope and Function
  - domain partitioning of, The Data Lake
  - modularity, Maintainability
    - deployability, Deployability
    - scalability and elasticity, Scalability
  - operational coupling, Sidecars and Service Mesh-When to Use
  - orchestrator per workflow, Orchestration Communication Style
  - saga pattern, Transactional Sagas
  - service-based architecture as stepping-stone, Component-Based Decomposition, Pattern Description, Pattern Description
  - shared database changed, Change control
  - single responsibility principle and granularity, Service Granularity, Service Scope and Function
- Microservices Patterns (Richardson), Transactional Sagas
- Microsoft SQL Server relational database, Relational Databases
- modularity
  - about, Architectural Modularity, Modularity Drivers
    - about change, Architectural Modularity
  - architectural governance, Using Fitness Functions
  - drivers of
    - about, Modularity Drivers
    - agility, Architectural Modularity, Modularity Drivers
    - deployability, Deployability
    - fault tolerance, Availability/Fault Tolerance
    - maintainability, Maintainability-Maintainability
    - scalability, Architectural Modularity, Scalability-Scalability
    - testability, Testability
  - granularity versus, Service Granularity
  - monolithic architectures
    - about modularity, Modularity Drivers
    - about need for modularity, Architectural Modularity
    - deployability, Deployability
    - fault tolerance, Availability/Fault Tolerance
    - maintainability, Maintainability
    - modular monoliths, Modularity Drivers
    - scalability and elasticity, Scalability

    - Sysops Squad saga, Architectural Modularity, Sysops Squad Saga: Creating a Business Case-Sysops Squad Saga: Creating a Business Case
    - testability, Testability
    - water glass analogy, Architectural Modularity
- monolithic architectures
    - architectural decomposition (see architectural decomposition)
    - Big Ball of Mud anti-pattern, Is the Codebase Decomposable?
        - tactical forking, Component-Based Decomposition-Trade-Offs
    - data decomposition
        - about, Decomposing Monolithic Data
        - assign tables to data domains, Step 2: Assign Tables to Data Domains-Step 2: Assign Tables to Data Domains
        - create data domains, Step 1: Analyze Database and Create Data Domains
        - Refactoring Databases (Ambler and Sadalage), Decomposing Monolithic Data
        - schemas to separate servers, Step 4: Move Schemas to Separate Database Servers
        - separate connections to data domains, Step 3: Separate Database Connections to Data Domains
        - switch over to independent servers, Step 5: Switch Over to Independent Database Servers
    - data importance, The Importance of Data in Architecture
    - database connection pool, Connection management
    - database type optimization, Database type optimization
    - Epic Saga mimicking, Epic Saga(sao) Pattern-Epic Saga(sao) Pattern
    - modular architecture
        - about need for, Architectural Modularity
        - deployability, Deployability
        - fault tolerance, Availability/Fault Tolerance
        - maintainability, Maintainability
        - modular monoliths, Modularity Drivers
        - scalability and elasticity, Scalability
        - Sysops Squad saga, Architectural Modularity, Sysops Squad Saga: Creating a Business Case-Sysops Squad Saga: Creating a Business Case
        - testability, Testability
        - water glass analogy, Architectural Modularity
    - namespaces prior to Domain-Driven Design, Pattern Description
    - patterns for migration to service-based
        - about, Component-Based Decomposition, Component-Based Decomposition Patterns
        - architecture stories, Component-Based Decomposition Patterns

    - create component domains, Create Component Domains Pattern-Fitness function: All namespaces under <root namespace node> should be restricted to <list of domains>
    - create domain services, Create Domain Services Pattern-Fitness function: All components in <some domain service> should start with the same namespace
    - determine component dependencies, Determine Component Dependencies Pattern-Fitness function: <some component> should not have a dependency on <another component>
    - flatten components, Flatten Components Pattern-Fitness function: No source code should reside in a root namespace
    - gather common domain components, Gather Common Domain Components Pattern-Fitness function: Find common code across components
    - identify and size components, Identify and Size Components Pattern-Sysops Squad Saga: Sizing Components
  - single architecture quantum, Independently Deployable
    - functional cohesion, High Functional Cohesion
    - static coupling, High Static Coupling
- Monorepo, Avoiding Snake Oil and Evangelism
- MySQL relational database, Relational Databases

N

- namespaces
  - component domains, Pattern Description
    - applications prior to Domain-Driven Design, Pattern Description
    - fitness functions for governance, Fitness Functions for Governance
  - components, Pattern Description
  - extend then prior a subdomain, Pattern Description
  - fitness function, Fitness function: All components in <some domain service> should start with the same namespace
  - root namespace definition, Pattern Description
- Neo4j graph database, Graph Databases
- NetArchTest tool, Using Fitness Functions
- network latency, Interservice Communication Pattern
- NewSQL databases, NewSQL Databases
- 97 Things Every Java Programmer Should Know (Henney and Gee), Code Replication
- NoSQL databases
  - aggregate orientation understood, Column Family Databases
  - column family databases, Column Family Databases

    - document databases, Document Databases
    - key-value databases, Key-Value Databases
    - NoSQL Distilled (Sadalage and Fowler), Key-Value Databases
    - schema-less databases, Document Databases
- Nygard, Michael, Architectural Decision Records

O

- online resources (see resources online)
- Online Transactional Processing (OLTP), The Importance of Data in Architecture
- operational coupling, Sidecars and Service Mesh-When to Use
    - Sysops Squad saga, Sysops Squad Saga: Common Infrastructure Logic
- operational data definition, The Importance of Data in Architecture
- Oracle Coherence replicated caching, Replicated Caching Pattern
- Oracle relational database, Relational Databases
- orchestrated coordination, Orchestration Communication Style-Orchestration Communication Style
    - about choreographed versus, Managing Distributed Workflows, Phone Tag Saga(sac) Pattern
        - trade-offs, Trade-Offs Between Orchestration and Choreography
    - about coordination, Coordination, Managing Distributed Workflows
    - definition, Architecture Versus Design: Keeping Definitions Simple
    - orchestrator, Orchestration Communication Style
    - workflow state management, Workflow State Management
- orchestrated request-based eventual consistency, Orchestrated Request-Based Pattern-Orchestrated Request-Based Pattern
- Orchestrated Saga pattern, Epic Saga(sao) Pattern-Epic Saga(sao) Pattern
- orphaned classes definition, Pattern Description
- orthogonal code reuse pattern, Sidecars and Service Mesh
- orthogonal coupling, Sidecars and Service Mesh
- out-of-context trap, The “Out-of-Context” Trap-The “Out-of-Context” Trap
- ownership of data
    - about, Data Ownership and Distributed Transactions
    - assigning, Assigning Data Ownership
    - common ownership, Common Ownership Scenario
    - data mesh domain ownership, Definition of Data Mesh
    - data sovereignty per service, Step 3: Separate Database Connections to Data Domains
    - distributed data access, Column Schema Replication Pattern, Replicated Caching Pattern

- joint ownership
  - about, Joint Ownership Scenario
  - data domain technique, Data Domain Technique-Data Domain Technique
  - delegate technique, Delegate Technique-Delegate Technique
  - table split technique, Table Split Technique-Table Split Technique
- service consolidation technique, Service Consolidation Technique
- single ownership, Single Ownership Scenario
- summary, Data Ownership Summary
- Sysops Squad saga, Data Ownership and Distributed Transactions, Sysops Squad Saga: Data Ownership for Ticket Processing-Sysops Squad Saga: Data Ownership for Ticket Processing

P

- Page-Jones, Meilir, Pulling Things Apart
- Parallel Saga pattern, Parallel Saga(aeo) Pattern-Parallel Saga(aeo) Pattern
- PCI (Payment Card Industry) data
  - security and granularity, Security
    - Sysops Squad saga, Sysops Squad Saga: Customer Registration Granularity-Sysops Squad Saga: Customer Registration Granularity
- Personally Identifiable Information (PII), The Data Lake
- Phone Tag Saga pattern, Phone Tag Saga(sac) Pattern-Phone Tag Saga(sac) Pattern
- PII (Personally Identifiable Information), The Data Lake
- Pipes and Filters design pattern, Time Travel Saga(sec) Pattern, Anthology Saga(aec) Pattern
- platforms for code reuse, Reuse via Platforms
- Ports and Adaptors Pattern, Sidecars and Service Mesh
- PostgreSQL relational database, Relational Databases
- publish-and-subscribe messaging, Event-Based Pattern
  - dead letter queue, Event-Based Pattern
  - durable subscribers, Event-Based Pattern
- pull model consumer-driven contracts, Consumer-driven contracts
- push model definition, Consumer-driven contracts

Q

- quanta, Architecture (Quantum | Quanta)
  - (see also architecture quantum)
- quorum mechanism, Key-Value Databases, Document Databases

R

- Redis key-value database, Key-Value Databases
- Refactoring Databases (Ambler and Sadalage), Decomposing Monolithic Data
- relational databases (RDBMS), Relational Databases-Relational Databases
  - links to most popular, Relational Databases
  - transactions and service granularity, Database Transactions
- replicate databases, Step 4: Move Schemas to Separate Database Servers
- replicated in-memory caching, Replicated Caching Pattern-Replicated Caching Pattern
- resources online
  - ArchUnit tool, Fitness function: All components in <some domain service> should start with the same namespace
  - book website, Using Code Examples
  - cohesion metrics information, Service Scope and Function
  - database links
    - cloud native databases, Cloud Native Databases
    - NoSQL databases, Key-Value Databases
    - relational databases, Relational Databases
  - dependency tools
    - JDepend, Using Fitness Functions, Afferent and Efferent Coupling
    - NetArchTest, Using Fitness Functions
    - visualization, Pattern Description
  - Refactoring Databases (Ambler and Sadalage), Decomposing Monolithic Data
  - Saga Pattern (Richardson), Transactional Sagas
  - stamp coupling information, Stamp Coupling
  - static code analysis tools, Pattern Description
- REST (representational state transfer) loose contracts, Strict Versus Loose Contracts
- reuse patterns (see code reuse patterns)
- Riak KV key-value database, Key-Value Databases
- Richards, Mark, Preface, Performance, Concept and Term References
- Richardson, Chris, Transactional Sagas
- root namespace definition, Pattern Description
- runtime versus compile-based changes, Change Risk

S

- Sadalage, Pramod, Decomposing Monolithic Data, Key-Value Databases
- sagas
  - about, Transactional Sagas

- definition, Introducing the Sysops Squad Saga
- dynamic coupling matrix, Coordination, Transactional Saga Patterns, Analyze Coupling Points
- microservice saga pattern, Transactional Sagas
- state machines, Saga State Machines-Saga State Machines
- Sysops Squad saga background, Introducing the Sysops Squad Saga
  - (see also Sysops Squad sagas)
- techniques for managing, Techniques for Managing Sagas
- transactional saga patterns
  - about, Coordination, Transactional Saga Patterns
  - about state management, State Management and Eventual Consistency
  - Anthology Saga, Anthology Saga(aec) Pattern-Anthology Saga(aec) Pattern
  - Epic Saga, Epic Saga(sao) Pattern-Epic Saga(sao) Pattern
  - Fairy Tale Saga, Fairy Tale Saga(seo) Pattern-Fairy Tale Saga(seo) Pattern
  - Fantasy Fiction Saga, Fantasy Fiction Saga(aao) Pattern-Fantasy Fiction Saga(aao) Pattern
  - Horror Story, Horror Story(aac) Pattern-Horror Story(aac) Pattern
  - Parallel Saga, Parallel Saga(aeo) Pattern-Parallel Saga(aeo) Pattern
  - Phone Tag Saga, Phone Tag Saga(sac) Pattern-Phone Tag Saga(sac) Pattern
  - Sysops Squad saga, Transactional Sagas, Sysops Squad Saga: Atomic Transactions and Compensating Updates-Sysops Squad Saga: Atomic Transactions and Compensating Updates
  - Time Travel Saga, Time Travel Saga(sec) Pattern-Time Travel Saga(sec) Pattern
- scalability
  - choreography, Workflow State Management, Phone Tag Saga(sac) Pattern
  - connection management, Scalability
    - database-per-service, Scalability
    - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
  - coupling relationship, Phone Tag Saga(sac) Pattern, Analyze Coupling Points
  - data disintegration driver, Scalability
  - database types
    - cloud native scalable, Cloud Native Databases
    - column family scalable, Column Family Databases
    - document scalable, Document Databases
    - graph read versus write, Graph Databases
    - key-value scalable, Key-Value Databases
    - NewSQL scalable, NewSQL Databases
    - relational complexity, Relational Databases
    - time series scalable, Time-Series Databases

    - definition, Scalability
    - elasticity versus, Sysops Squad Saga: Understanding Quanta, Scalability
    - granularity disintegration driver, Scalability and Throughput
    - modularity providing, Architectural Modularity, Scalability
        - modularity driver, Modularity Drivers, Scalability-Scalability
    - orchestration, Orchestration Communication Style
        - choreography versus, Phone Tag Saga(sac) Pattern
    - service consolidation for data ownership, Service Consolidation Technique
    - shared services, Scalability
- schemas for database tables, Step 2: Assign Tables to Data Domains
    - cross-schema access resolved, Step 3: Separate Database Connections to Data Domains
    - data domain versus data schema, Step 2: Assign Tables to Data Domains
    - databases separated physically, Step 4: Move Schemas to Separate Database Servers
    - schema-less databases, Document Databases
    - synonyms for tables, Step 2: Assign Tables to Data Domains
- Scylla column family database, Column Family Databases
- security
    - granularity disintegration driver, Security
    - PCI data Sysops Squad saga, Sysops Squad Saga: Customer Registration Granularity-Sysops Squad Saga: Customer Registration Granularity
    - security latency, Interservice Communication Pattern
- semantic coupling, Choreography Communication Style
    - implementation coupling, Choreography Communication Style
    - stamp coupling to manage workflow, Stamp Coupling for Workflow Management
- service consolidation technique of joint ownership, Service Consolidation Technique
- service granularity (see granularity)
- service mesh, Sidecars and Service Mesh-When to Use
    - Sysops Squad saga, Sysops Squad Saga: Common Infrastructure Logic
- service-based architecture
    - about, High Static Coupling, Component-Based Decomposition, Pattern Description
    - data access in other domains, Step 3: Separate Database Connections to Data Domains
    - data shared by services, Pulling Apart Operational Data
        - changes to database, Change control
        - connection management, Connection management-Connection management

    - single point of failure, Fault tolerance
  - domain services from component domains, Pattern Description
    - Create Domain Services pattern, Create Domain Services Pattern-Fitness function: All components in <some domain service> should start with the same namespace
  - evolution of architecture, Giving Timeless Advice About Software Architecture
  - modular architecture scalability and elasticity, Scalability
  - monolithic migration to
    - about, Component-Based Decomposition, Component-Based Decomposition Patterns
    - architecture stories, Component-Based Decomposition Patterns
    - create component domains, Create Component Domains Pattern-Fitness function: All namespaces under <root namespace node> should be restricted to <list of domains>
    - create domain services, Create Domain Services Pattern-Fitness function: All components in <some domain service> should start with the same namespace
    - determine component dependencies, Determine Component Dependencies Pattern-Fitness function: <some component> should not have a dependency on <another component>
    - flatten components pattern, Flatten Components Pattern-Fitness function: No source code should reside in a root namespace
    - gather common domain components, Gather Common Domain Components Pattern-Fitness function: Find common code across components
    - identify and size components, Identify and Size Components Pattern-Sysops Squad Saga: Sizing Components
  - static coupling, High Static Coupling
  - stepping-stone to microservices, Component-Based Decomposition, Pattern Description, Pattern Description
- services
  - bounded contexts (see bounded context)
  - choreographed coordination definition, Architecture Versus Design: Keeping Definitions Simple
  - component-based decomposition, Component-Based Decomposition
    - Create Component Domains pattern, Create Component Domains Pattern
  - coupling definition, Architecture Versus Design: Keeping Definitions Simple
    - (see also coupling)
  - data access in other domains, Step 3: Separate Database Connections to Data Domains
  - data shared by services, Pulling Apart Operational Data

    - changes to database, Change control
    - connection management, Connection management-Connection management
    - single point of failure, Fault tolerance
- database connection pool, Connection management
    - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
- database-per-service, Scalability
- definition, Architecture Versus Design: Keeping Definitions Simple
- distributed data access
    - Column Schema Replication pattern, Column Schema Replication Pattern
    - Data Domain pattern, Data Domain Pattern-Data Domain Pattern
    - Interservice Communication Pattern, Interservice Communication Pattern
    - Replicated Caching pattern, Replicated Caching Pattern-Replicated Caching Pattern
    - Sysops Squad saga, Distributed Data Access, Sysops Squad Saga: Data Access for Ticket Assignment-Sysops Squad Saga: Data Access for Ticket Assignment
- extensibility, Extensibility
- granularity (see granularity)
- identifying in sagas, Techniques for Managing Sagas
- modularity
    - deployability, Deployability
    - scalability and elasticity, Scalability
    - testability, Testability
- orchestrated coordination definition, Architecture Versus Design: Keeping Definitions Simple
- ownership of data
    - assigning, Assigning Data Ownership
    - common ownership, Common Ownership Scenario
    - data sovereignty per service, Step 3: Separate Database Connections to Data Domains
    - distributed data access, Column Schema Replication Pattern, Replicated Caching Pattern
    - joint ownership, Joint Ownership Scenario-Delegate Technique
    - service consolidation technique, Service Consolidation Technique
    - single ownership, Single Ownership Scenario
    - summary, Data Ownership Summary
    - Sysops Squad saga, Data Ownership and Distributed Transactions, Sysops Squad Saga: Data Ownership for Ticket Processing-Sysops Squad Saga: Data Ownership for Ticket Processing

    - service mesh, Sidecars and Service Mesh-When to Use
    - shared services for code reuse, Shared Service-When to Use
    - Sidecar pattern, Sidecars and Service Mesh-When to Use
- sharding in databases, Key-Value Databases
    - document databases, Document Databases
    - NewSQL databases, NewSQL Databases
- shared code and granularity, Shared Code-Shared Code
    - (see also code reuse patterns)
    - shared libraries, Dependency Management and Change Control
- shared domain functionality
    - granularity and, Shared Code
    - shared infrastructure functionality versus, Pattern Description
    - Sysops Squad saga, Sysops Squad Saga: Shared Domain Functionality
- shared libraries
    - about, Shared Library
    - fault tolerance, Fault Tolerance
    - granularity and, Dependency Management and Change Control
    - versioning strategies, Versioning Strategies-When To Use
- shared services for code reuse, Shared Service-When to Use
    - change risk, Change Risk
    - fault tolerance, Fault Tolerance
    - performance, Performance
    - scalability, Scalability
    - Sysops Squad saga, Sysops Squad Saga: Shared Domain Functionality
    - versioning via API endpoints, Change Risk
- side effects in distributed architectures, Sysops Squad Saga: Atomic Transactions and Compensating Updates
- Sidecar pattern, Sidecars and Service Mesh-When to Use
    - Sysops Squad saga, Sysops Squad Saga: Common Infrastructure Logic
- silver-bullet solutions, What Happens When There Are No “Best Practices”?
- single point of failure (SPOF), Fault tolerance
    - Sysops Squad saga, Sysops Squad Saga: Justifying Database Decomposition
- single responsibility principle, Service Scope and Function
    - granularity and, Service Granularity, Service Scope and Function
- SMS information link, Service Scope and Function
- Snowflake cloud native database, Cloud Native Databases
- soft state of BASE transactions, Distributed Transactions
- software architecture (see architecture)
- SOLID principles (Martin), Service Scope and Function
- SonarQube code hygiene tool, Using Fitness Functions

- speed-to-market as modularity driver, Modularity Drivers
- stamp coupling
  - about, Stamp Coupling
  - bandwidth, Bandwidth
  - choreography state management, Workflow State Management, Phone Tag Saga(sac) Pattern
  - over-coupling via, Over-Coupling via Stamp Coupling
  - workflow management, Stamp Coupling for Workflow Management
- standard deviation calculation, Fitness function: No component shall exceed <some number of standard deviations> from the mean component size
- Star Schema of data warehouses, The Data Warehouse, The Data Warehouse
- Starbucks Does Not Use Two-Phase Commit (Hohpe), Sysops Squad Saga: Atomic Transactions and Compensating Updates
- state machines, Saga State Machines-Saga State Machines
- state management
  - compensating updates versus, Saga State Machines
  - finite state machines for, State Management and Eventual Consistency
    - about state machines, Saga State Machines-Saga State Machines
  - orchestration, Orchestration Communication Style
  - stamp coupling for workflow management, Stamp Coupling for Workflow Management
  - workflow state management, Workflow State Management-Workflow State Management
- stateless choreography, Workflow State Management
- statements counted, Pattern Description, Service Granularity
- static code analysis tools resource, Pattern Description
- static coupling, Pulling Things Apart, Architecture (Quantum | Quanta)
  - architecture quantum, High Static Coupling
    - high static coupling, High Static Coupling-High Static Coupling
    - independently deployable, Independently Deployable
    - Sysops Squad saga, Sysops Squad Saga: Understanding Quanta
  - dynamic versus, Architecture (Quantum | Quanta)
    - Sysops Squad saga, Sysops Squad Saga: Understanding Quanta
  - operational dependencies as, Pulling Things Apart, Architecture (Quantum | Quanta)
  - trade-off analysis, Coupling
    - coupling points, Analyze Coupling Points
    - static coupling diagram, Coupling
- Stine, Matt, Deployability
- strict contracts, Strict Versus Loose Contracts

    - JSON example, Strict Versus Loose Contracts
    - trade-offs, Strict contracts
- Structured Design (Yourdon and Constantine), Afferent and Efferent Coupling
- survey database structure, Sysops Squad Saga: Polyglot Databases-Sysops Squad Saga: Polyglot Databases
- synchronous communication
    - asynchronous versus, Communication, Sysops Squad Saga: Understanding Quanta
    - definition, Architecture Versus Design: Keeping Definitions Simple
    - delegate technique of data ownership, Delegate Technique
    - fault tolerance, Availability/Fault Tolerance
    - granularity and, Workflow and Choreography-Workflow and Choreography
    - scalability and elasticity versus, Scalability
- synonyms for tables, Step 2: Assign Tables to Data Domains
- Sysops Squad sagas
    - about, Introducing the Sysops Squad Saga
        - nonticketing workflow, Nonticketing Workflow
        - problem, A Bad Scenario
        - ticketing workflow, Ticketing Workflow
    - analytical data, Managing Analytical Data, The Data Warehouse, The Data Lake
        - data mesh, Sysops Squad Saga: Data Mesh-Sysops Squad Saga: Data Mesh
    - architectural components, Sysops Squad Architectural Components
    - architecture quantum, Sysops Squad Saga: Understanding Quanta
    - contracts, Contracts, Sysops Squad Saga: Managing Ticketing Contracts
    - data model, Sysops Squad Data Model
        - ACID transaction example, Distributed Transactions
        - data domains, Decomposing Monolithic Data
    - distributed data access, Distributed Data Access, Sysops Squad Saga: Data Access for Ticket Assignment-Sysops Squad Saga: Data Access for Ticket Assignment
        - Column Schema Replication pattern, Column Schema Replication Pattern
        - Data Domain pattern, Data Domain Pattern-Data Domain Pattern
        - Interservice Communication Pattern, Interservice Communication Pattern
        - Replicated Caching pattern, Replicated Caching Pattern-Replicated Caching Pattern
    - distributed transactions, Sysops Squad Saga: Data Ownership for Ticket Processing-Sysops Squad Saga: Data Ownership for Ticket Processing
    - distributed workflows, Managing Distributed Workflows, Sysops Squad Saga: Managing Workflows-Sysops Squad Saga: Managing Workflows
    - epilogue, Sysops Squad Saga: Epilogue

- eventual consistency
  - about, Eventual Consistency Patterns
  - background synchronization, Background Synchronization Pattern-Background Synchronization Pattern
  - event-based pattern, Event-Based Pattern-Event-Based Pattern
  - orchestrated request-based pattern, Orchestrated Request-Based Pattern-Orchestrated Request-Based Pattern
- monolithic application broken apart, Architectural Modularity
  - 6. create domain services, Pattern Description
  - business case for, Sysops Squad Saga: Creating a Business Case-Sysops Squad Saga: Creating a Business Case
  - code reuse patterns, Reuse Patterns, Sysops Squad Saga: Common Infrastructure Logic, Sysops Squad Saga: Shared Domain Functionality
  - create component domains, Sysops Squad Saga: Creating Component Domains-Sysops Squad Saga: Creating Component Domains
  - create domain services, Sysops Squad Saga: Creating Domain Services
  - data pulled apart, Pulling Apart Operational Data, Sysops Squad Saga: Justifying Database Decomposition
  - database structure, Sysops Squad Saga: Polyglot Databases-Sysops Squad Saga: Polyglot Databases
  - database types, Sysops Squad Saga: Polyglot Databases
  - decomposition method, Architectural Decomposition, Sysops Squad Saga: Choosing a Decomposition Approach
  - decomposition pattern, Component-Based Decomposition Patterns
  - determine component dependencies, Sysops Squad Saga: Identifying Component Dependencies-Sysops Squad Saga: Identifying Component Dependencies
  - flatten components, Sysops Squad Saga: Flattening Components-Sysops Squad Saga: Flattening Components
  - gather common domain components, Sysops Squad Saga: Gathering Common Components-Sysops Squad Saga: Gathering Common Components
  - granularity, Service Granularity, Sysops Squad Saga: Ticket Assignment Granularity-Sysops Squad Saga: Customer Registration Granularity
  - identify and size components, Sysops Squad Saga: Sizing Components-Sysops Squad Saga: Sizing Components
  - ownership of data, Data Ownership and Distributed Transactions, Sysops Squad Saga: Data Ownership for Ticket Processing-Sysops Squad Saga: Data Ownership for Ticket Processing
- state machines, Saga State Machines-Saga State Machines

- trade-off analysis, Build Your Own Trade-Off Analysis, Sysops Squad Saga: Epilogue
- transactional sagas, Transactional Sagas, Sysops Squad Saga: Atomic Transactions and Compensating Updates-Sysops Squad Saga: Atomic Transactions and Compensating Updates

T

- table split technique of joint ownership, Table Split Technique-Table Split Technique
- tactical forking, Component-Based Decomposition-Trade-Offs
  - about, Architectural Decomposition
  - trade-offs, Trade-Offs
- technical partitioning
  - data warehouses as, The Data Warehouse
  - domain partitioning preferred, The Data Lake
  - domain workflows, Choreography Communication Style
  - maintainability and, Maintainability
- testability as modularity driver, Testability
- Thoughtworks
  - Architectural Decision Records, Architectural Decision Records
  - composition versus inheritance, Shared Service
- time series databases, Time-Series Databases-Time-Series Databases
- Time Travel Saga pattern, Time Travel Saga(sec) Pattern-Time Travel Saga(sec) Pattern
- TimeScale time series database, Time-Series Databases
- tools
  - ArchUnit, Fitness function: All components in <some domain service> should start with the same namespace
  - dependencies
    - JDepend, Using Fitness Functions, Afferent and Efferent Coupling
    - NetArchTest, Using Fitness Functions
    - visualization, Afferent and Efferent Coupling, Pattern Description, Pattern Description
  - SonarQube code hygiene tool, Using Fitness Functions
  - static code analysis, Pattern Description
  - visualization
    - about importance of, Pattern Description
    - afferent and efferent coupling, Afferent and Efferent Coupling
    - data domains via soccer ball, Decomposing Monolithic Data

    - dependencies, Afferent and Efferent Coupling, Pattern Description, Pattern Description
    - distance from main sequence, Distance from the Main Sequence
    - domain diagramming exercise, Sysops Squad Saga: Creating Component Domains
    - graph databases, Graph Databases-Graph Databases
    - isomorphic diagrams, Transactional Saga Patterns
    - static coupling diagram, Coupling
- trade-off analysis, Discerning Coupling in Software Architecture-Discerning Coupling in Software Architecture
    - assessing trade-offs, Assess Trade-Offs
        - least worst combination, What Happens When There Are No “Best Practices”?
    - building your own, Build Your Own Trade-Off Analysis-Analyze Coupling Points
    - consistency and availability, Time-Series Databases
    - coordination, Trade-Offs Between Orchestration and Choreography
    - coupling, Discerning Coupling in Software Architecture-Discerning Coupling in Software Architecture, Coupling-Analyze Coupling Points
        - coupling points, Analyze Coupling Points
        - dynamic quantum coupling factors, Coordination
        - static coupling diagram, Coupling
    - documenting versus governing, Architectural Decision Records
    - granularity, Finding the Right Balance
    - iterative nature of, Analyze Coupling Points
    - loose contracts, Loose contracts
    - strict contracts, Strict contracts
    - Sysops Squad saga, Build Your Own Trade-Off Analysis, Sysops Squad Saga: Epilogue
    - tactical forking, Trade-Offs
    - techniques of
        - bottom line decision, Prefer Bottom Line over Overwhelming Evidence
        - MECE lists, MECE Lists
        - modeling relevant domain cases, Model Relevant Domain Cases-Model Relevant Domain Cases
        - out-of-context trap, The “Out-of-Context” Trap-The “Out-of-Context” Trap
        - qualitative versus quantitative analysis, Qualitative Versus Quantative Analysis
        - snake oil and evangelism, Avoiding Snake Oil and Evangelism
- transactional saga patterns
    - about, Transactional Saga Patterns

    - pattern matrix, Coordination, Transactional Saga Patterns, Analyze Coupling Points
  - about sagas, Transactional Sagas
    - management techniques, Techniques for Managing Sagas
  - about state management, State Management and Eventual Consistency
    - about state machines, Saga State Machines-Saga State Machines
    - compensating updates versus, Saga State Machines
  - Anthology Saga, Anthology Saga(aec) Pattern-Anthology Saga(aec) Pattern
  - Epic Saga, Epic Saga(sao) Pattern-Epic Saga(sao) Pattern
  - Fairy Tale Saga, Fairy Tale Saga(seo) Pattern-Fairy Tale Saga(seo) Pattern
  - Fantasy Fiction Saga, Fantasy Fiction Saga(aao) Pattern-Fantasy Fiction Saga(aao) Pattern
  - Horror Story, Horror Story(aac) Pattern-Horror Story(aac) Pattern
  - Parallel Saga, Parallel Saga(aeo) Pattern-Parallel Saga(aeo) Pattern
  - Phone Tag Saga, Phone Tag Saga(sac) Pattern-Phone Tag Saga(sac) Pattern
  - Sysops Squad saga, Transactional Sagas, Sysops Squad Saga: Atomic Transactions and Compensating Updates-Sysops Squad Saga: Atomic Transactions and Compensating Updates
  - Time Travel Saga, Time Travel Saga(sec) Pattern-Time Travel Saga(sec) Pattern
- transactionality
  - distributed architecture challenge, Consistency
  - security versus, Sysops Squad Saga: Customer Registration Granularity-Sysops Squad Saga: Customer Registration Granularity
- transactions
  - data integration driver, Database transactions
  - database transactions and granularity, Database Transactions
  - distributed transactions
    - about, Distributed Transactions, Epic Saga(sao) Pattern
    - ACID transactions, Distributed Transactions-Distributed Transactions
    - BASE transactions, Distributed Transactions
    - compensating updates, Orchestrated Request-Based Pattern, Epic Saga(sao) Pattern
    - eventual consistency, Eventual Consistency Patterns
      - (see also eventual consistency)
    - Sysops Squad saga, Sysops Squad Saga: Data Ownership for Ticket Processing-Sysops Squad Saga: Data Ownership for Ticket Processing, Sysops Squad Saga: Atomic Transactions and Compensating Updates-Sysops Squad Saga: Atomic Transactions and Compensating Updates
- Trunk-based Development, Avoiding Snake Oil and Evangelism
- tunable consistency, Key-Value Databases, Column Family Databases

U

- user interface quantum, High Static Coupling

V

- version control systems
  - code change frequency metric, Code Volatility
  - LATEST caution, Versioning Strategies
  - shared code libraries, Versioning Strategies-When To Use
    - granularity and, Shared Code, Shared Code
  - shared service API endpoint versioning, Change Risk
  - strict contracts requiring, Strict contracts
- visualization tools
  - afferent and efferent coupling, Afferent and Efferent Coupling
  - data domains via soccer ball, Decomposing Monolithic Data
  - dependencies, Pattern Description
    - importance of visualization, Pattern Description
    - JDepend Eclipse plug-in, Afferent and Efferent Coupling
  - distance from main sequence, Distance from the Main Sequence
  - domain diagramming exercise, Sysops Squad Saga: Creating Component Domains
  - graph databases, Graph Databases-Graph Databases
  - isomorphic diagrams, Transactional Saga Patterns
  - static coupling diagram, Coupling
- volatility-based decomposition, Code Volatility

W

- What Every Programmer Should Know About Object-Oriented Design (Page-Jones), Pulling Things Apart
- why more important than how, Architecture Versus Design: Keeping Definitions Simple
- wide column databases, Column Family Databases
- Woolf, Bobby, Step 1: Analyze Database and Create Data Domains
- workflow and service granularity, Workflow and Choreography-Workflow and Choreography
  - (see also distributed workflows)

Y

- Yourdon, Edward, Afferent and Efferent Coupling

Z

- zero-day exploits, Using Fitness Functions
- Zitzewitz, Alexander von, Maintainability
- zones of uselessness and pain, Distance from the Main Sequence

Previous chapter

< C. Trade-Off References

Next chapter

About the Authors >